cindyakinyi / phase3project

Code Issues Pull requests Actions Projects Wiki Security Ins

phase3project / indexp3.ipynb

cindyakinyi Initial commit phase 3 project 44f3bb1 · 6 minutes ago

386 lines (386 loc) · 13.5 KB

Preview Code Blame Raw

STUDENT NAME : CINDY AKINYIs.

PROJECT OBJECTIVE

The goal of this project was to predict **customer churn** in a telecommunications dataset — determining whether a customer is likely to leave the company.
This is a **binary classification problem**, where understanding *why* customers churn is as important as accurately predicting it. ns.

1. IMPORT LIBRARIES AND LOAD NECESSARY DATASETS

In [ ]:

```
# step 1: import libraries
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.tree import DecisionTreeClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import GridSearchCV
from sklearn.metrics import classification_report, confusion_matrix, accuracy_s
```

In [ ]:

```
#  Step 2: Load Dataset
df = pd.read_csv('syriatel.csv', index_col=0)
print(df.head())
print(df.info())
```

2. BASIC DATA CLEANING

Drop irrelevant column,and remove duplicatess

In [ ]:

```
# Step 3: Basic Data Cleaning
df.dropna(axis=1, how='all', inplace=True) # Drop empty columns
df.drop_duplicates(inplace=True) # Remove duplicates


# Drop irrelevant or identifier columns if present
for col in df.columns:
    if 'phone' in col.lower() or 'number' in col.lower() or 'id' in col.lower()
        df.drop(columns=[col], inplace=True, errors='ignore')
```

3. ENCODING AND FEATURE ENGINEERING

In [ ]:

```
#  Step 4: Encode Categorical Features
if 'international plan' in df.columns:
    df['international plan'] = df['international plan'].map({'yes': 1, 'no': 0}
if 'voice mail plan' in df.columns:
    df['voice mail plan'] = df['voice mail plan'].map({'yes': 1, 'no': 0})
if 'churn' in df.columns:
    df['churn'] = df['churn'].map({True: 1, False: 0, 'yes': 1, 'no': 0})
# One-hot encode any remaining categorical columns
df = pd.get_dummies(df, drop_first=True)
#  Feature Engineering - total usage metrics
if {'total day minutes', 'total eve minutes', 'total night minutes'}.issubset(c
    df['total_mins'] = (
        df['total day minutes'] + df['total eve minutes'] + df['total night mir
    )

if {'total day charge', 'total eve charge', 'total night charge'}.issubset(df.c
    df['total_charge'] = (
        df['total day charge'] + df['total eve charge'] + df['total night charg
    )

print(" Added feature-engineered columns: total_mins, total_charge")
```

4. FEATURE PREPARATION

In [ ]:

```
#  Step 5: Feature and Target Split
y = df['churn']
X = df.drop(columns=['churn'], errors='ignore')
```

In [ ]:

```
#  Step 6: Train-Test Split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random
```

In [ ]:

```
#  Step 7: Feature Scaling
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
```

MODEL TRAINING AND EVALUATION

In [ ]:

```
#  Step 8: Model Training and Evaluation
# decision tree
dt_model = DecisionTreeClassifier(random_state=42)
dt_model.fit(X_train, y_train)
y_pred_dt = dt_model.predict(X_test)

print('\n Decision Tree Results')
print('Accuracy:', round(accuracy_score(y_test, y_pred_dt) * 100, 2), '%')
print('\nConfusion Matrix:\n', confusion_matrix(y_test, y_pred_dt))
```

```
print("\nClassification Report:\n", classification_report(y_test, y_pred_dt))
```

In [ ]:

```
# 🔧 Hyperparameter Tuning for Decision Tree (GridSearchCV)
# Define parameter grid for tuning
param_grid = {
    'max_depth': [None, 5, 10, 20],
    'min_samples_split': [2, 5, 10],
    'criterion': ['gini', 'entropy']
}

# Initialize GridSearchCV
grid_dt = GridSearchCV(
    DecisionTreeClassifier(random_state=42),
    param_grid=param_grid,
    cv=3,
    scoring='accuracy',
    n_jobs=-1
)

# Fit to training data (use scaled data only if your dataset required scaling)
grid_dt.fit(X_train, y_train)

# Print results
print("\n Best Parameters from Decision Tree GridSearchCV:")
print(grid_dt.best_params_)
print("Best Cross-Validation Accuracy:", round(grid_dt.best_score_ * 100, 2), '

# Save best model
best_dt_model = grid_dt.best_estimator_
```

In [ ]:

```
#  Evaluate the Tuned Decision Tree Model on Test Data
# Make predictions using the best tuned model
y_pred_best_dt = best_dt_model.predict(X_test)

# Print evaluation metrics
print("\n Tuned Decision Tree Model Evaluation Results")
print("Test Accuracy:", round(accuracy_score(y_test, y_pred_best_dt) * 100, 2),
print("\nConfusion Matrix:\n", confusion_matrix(y_test, y_pred_best_dt))
print("\nClassification Report:\n", classification_report(y_test, y_pred_best_d
```

In [ ]:

```
# Logistic Regression
log_model = LogisticRegression(max_iter=1000, random_state=42)
log_model.fit(X_train_scaled, y_train)
y_pred_log = log_model.predict(X_test_scaled)


print('\n📈 Logistic Regression Results')
print('Accuracy:', accuracy_score(y_test, y_pred_log))
print('Confusion Matrix:\n', confusion_matrix(y_test, y_pred_log))
print('Classification Report:\n', classification_report(y_test, y_pred_log))
```

6. VISUAL EVALUATION

In [ ]:

```
# Step 9: Visual Evaluation
plt.figure(figsize=(6, 4))
sns.heatmap(confusion_matrix(y_test, y_pred_best_dt), annot=True, fmt='d', cmap
plt.title(' Decision Tree Confusion Matrix')
plt.xlabel('Predicted')
plt.ylabel('Actual')
plt.show()
```

SUMMARY

Models Developed

Two models were implemented and compared:

- **Decision Tree Classifier** — chosen for its ability to capture complex, nonlinear relationships between features and automatically select the most important predictors.
- **Logistic Regression** — used as a baseline for comparison because of its simplicity, interpretability, and ability to show how each variable contributes to churn risk.

📊 Model Performance

The **Decision Tree model** performed exceptionally well, achieving an accuracy of approximately **97.26%**. It effectively identified key churn drivers such as:

- High **customer service call frequency**
- Increased **total day minutes and charges**
- Presence of an **international plan**

These results suggest that heavy users and customers who frequently contact support are more likely to churn, possibly due to dissatisfaction or cost concerns.

The **Logistic Regression model** achieved an accuracy of **86.3%**, offering interpretability and a clearer understanding of the relationships between features and churn likelihood. Although slightly less accurate, it provided valuable insight into the direction and magnitude of feature impas.

💡 Insights and Implications

- **Customer service interactions** are a strong signal of dissatisfaction and should be monitored closely.
- **High usage customers** may face billing or quality issues that drive churn.
- **International plan users** may require tailored offers or communication to reduce

cancellations.

By combining both models, the company gains a balance between **predictive power** and **interpretability**, which strengthens data-driven decion-making.

⚠️ Limitations

- The dataset may have **class imbalance**, leading to lower recall for minority (churn) cases.
- Only basic scaling and tuning were applied; advanced techniques such as **SMOTE**, **cross-validation**, or **ensemble stacking** could improve results.
- The Decision Tree's interpretability is limited compared to siler linear models.

🚀 Recommendations

1. **Develop targeted retention campaigns** for customers with high call frequencies and heavy usage.
2. **Improve service quality monitoring** to address common issues identified in customer complaints.